

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]

03

Database Storage –Part I



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Homework #1 is due September 11th @ 11:59pm

Project #1 will be released on September 11th

OVERVIEW

We now understand what a database looks like at a logical level and how to write queries to read/write data from it.

We will next learn how to build software that manages a database.

COURSE OUTLINE

Relational Databases

Storage

Execution

Concurrency Control

Recovery

Distributed Databases

Potpourri

Query Planning

Operator Execution

Access Methods

Buffer Pool Manager

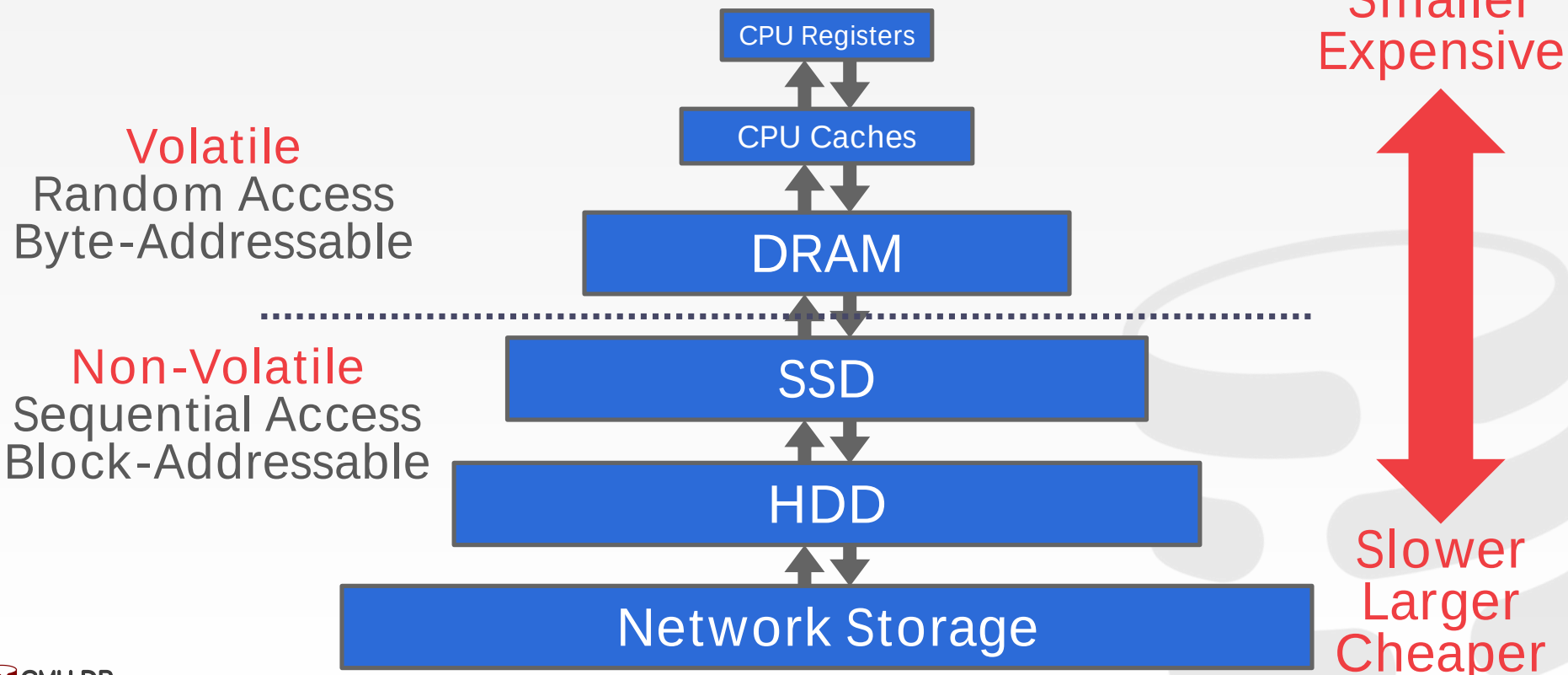
Disk Manager

DISK-ORIENTED ARCHITECTURE

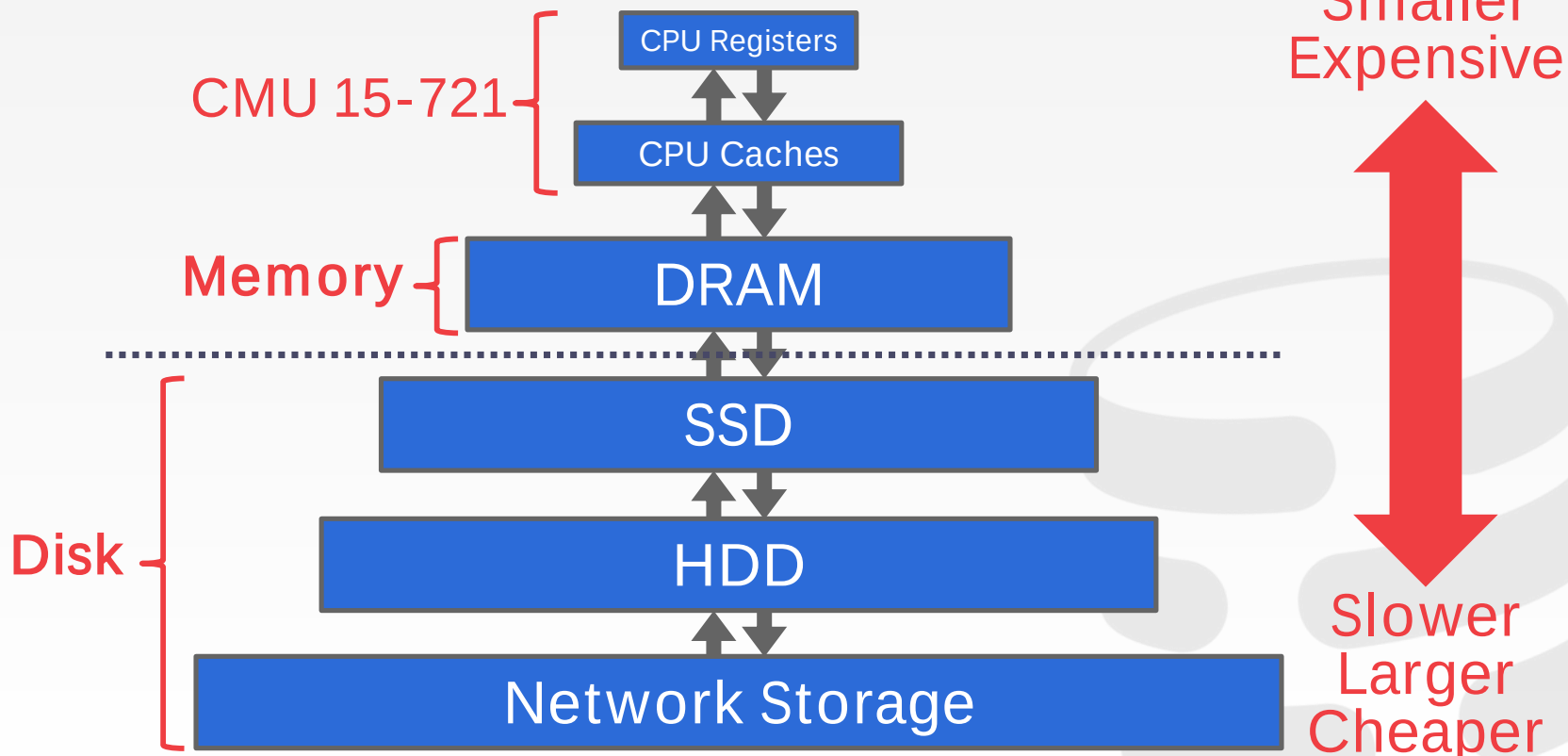
The DBMS assumes that the primary storage location of the database is on non-volatile disk.

The DBMS's components manage the movement of data between non-volatile and volatile storage.

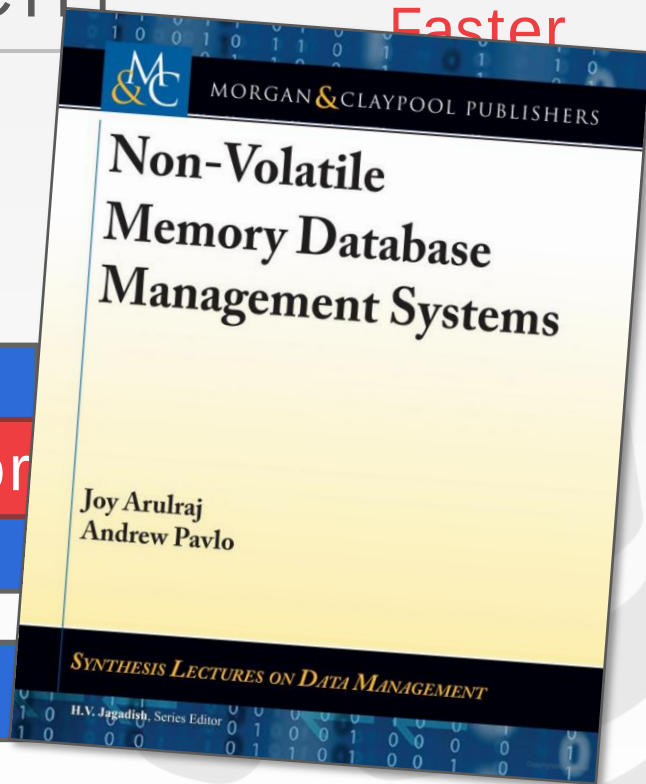
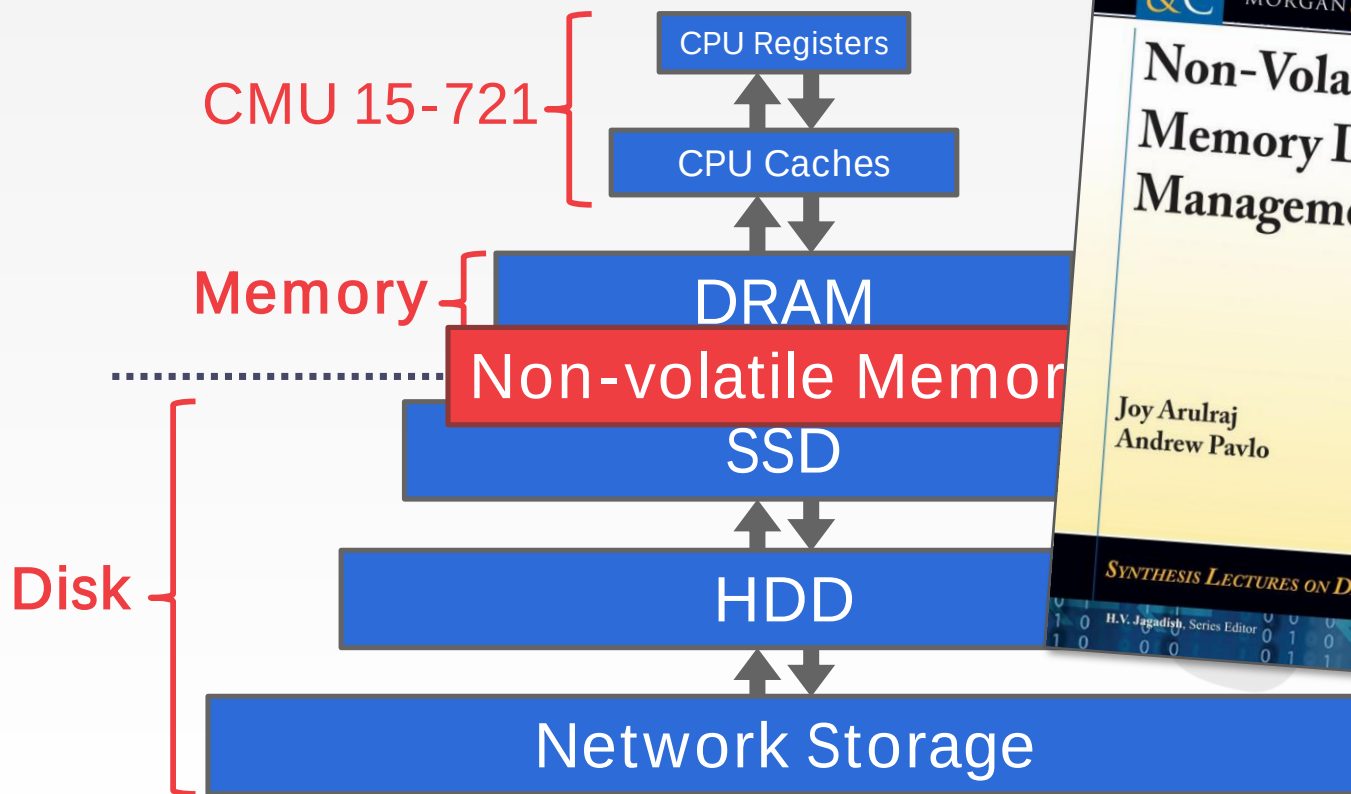
STORAGE HIERARCHY



STORAGE HIERARCHY



STORAGE HIERARCHY



**Larger
Cheaper**

ACCESS TIMES

0.5 ns L1 Cache Ref	← 0.5 sec
7 ns L2 Cache Ref	← 7 sec
100 ns DRAM	← 100 sec
150,000 ns SSD	← 1.7 days
10,000,000 ns HDD	← 16.5 weeks
~30,000,000 ns Network Storage	← 11.4 months
1,000,000,000 ns Tape Archives	← 31.7 years

SYSTEM DESIGN GOALS

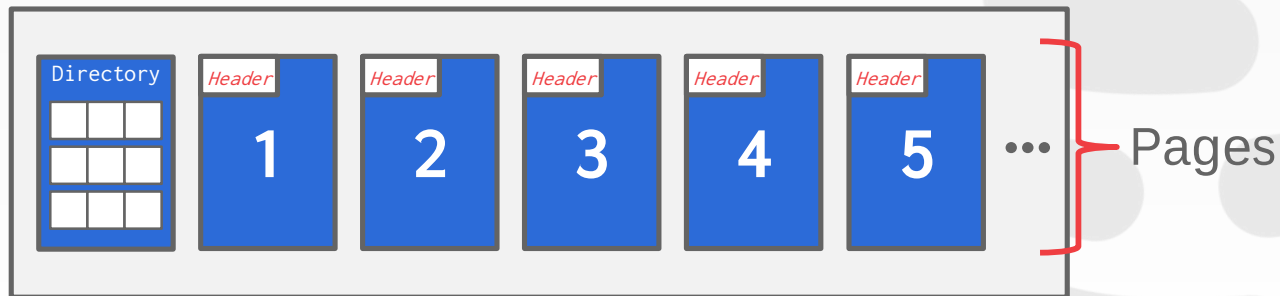
Allow the DBMS to manage databases that exceed the amount of memory available.

Reading/writing to disk is expensive, so it must be managed carefully to avoid large stalls and performance degradation.

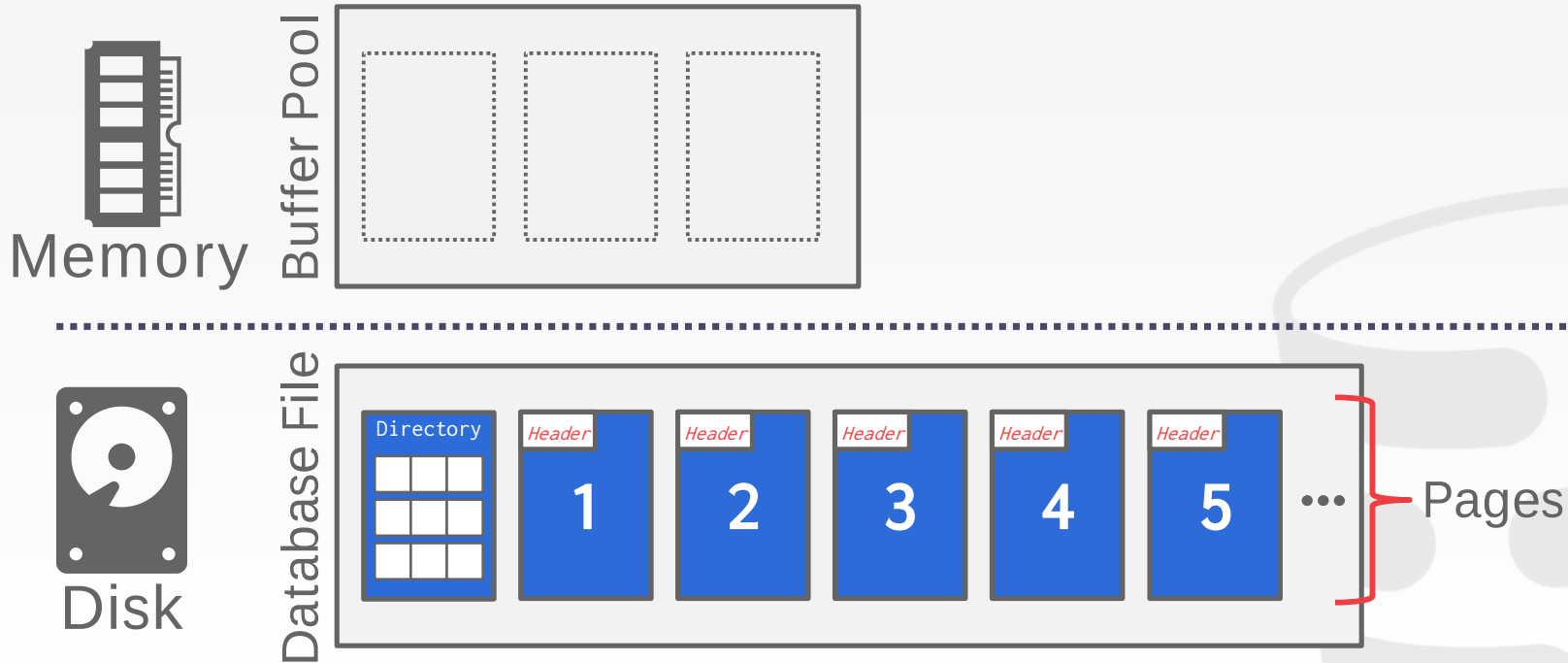
DISK-ORIENTED DBMS



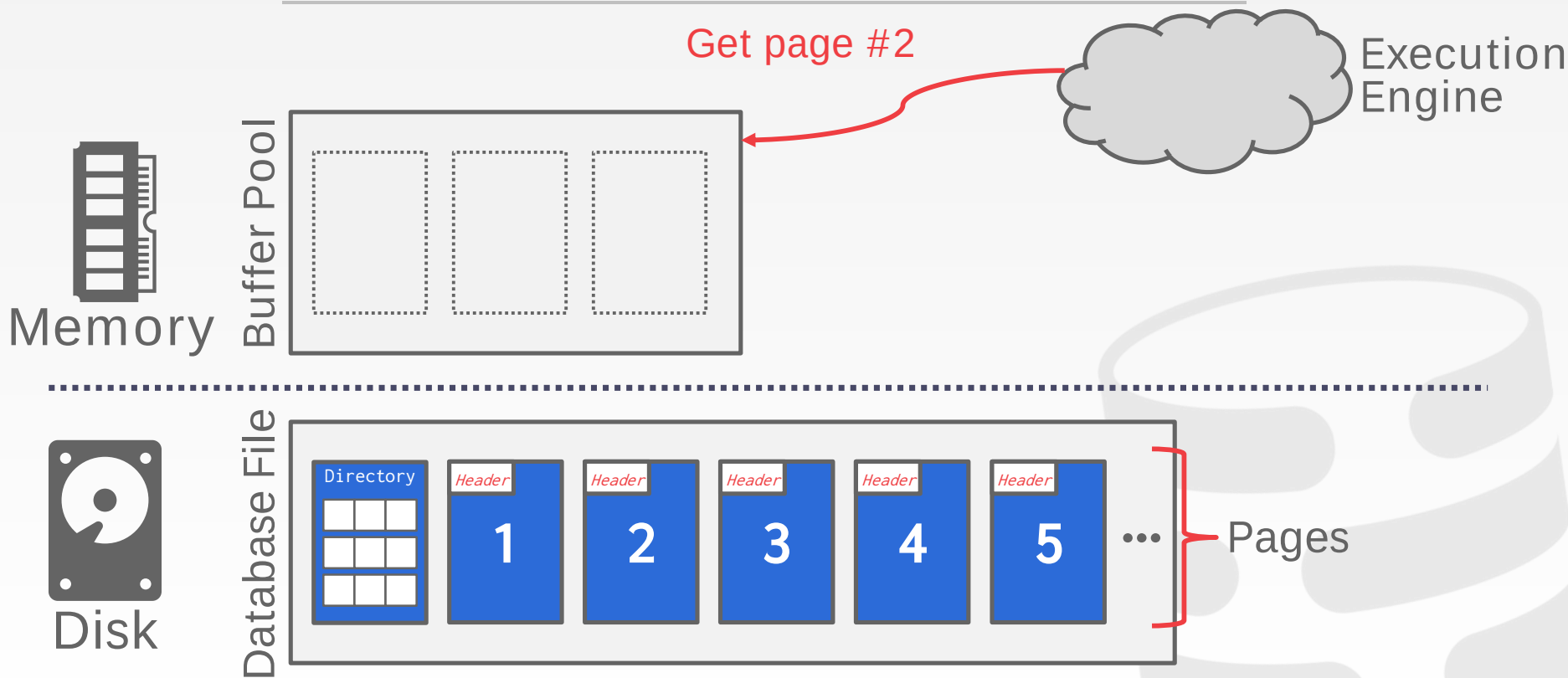
Database File



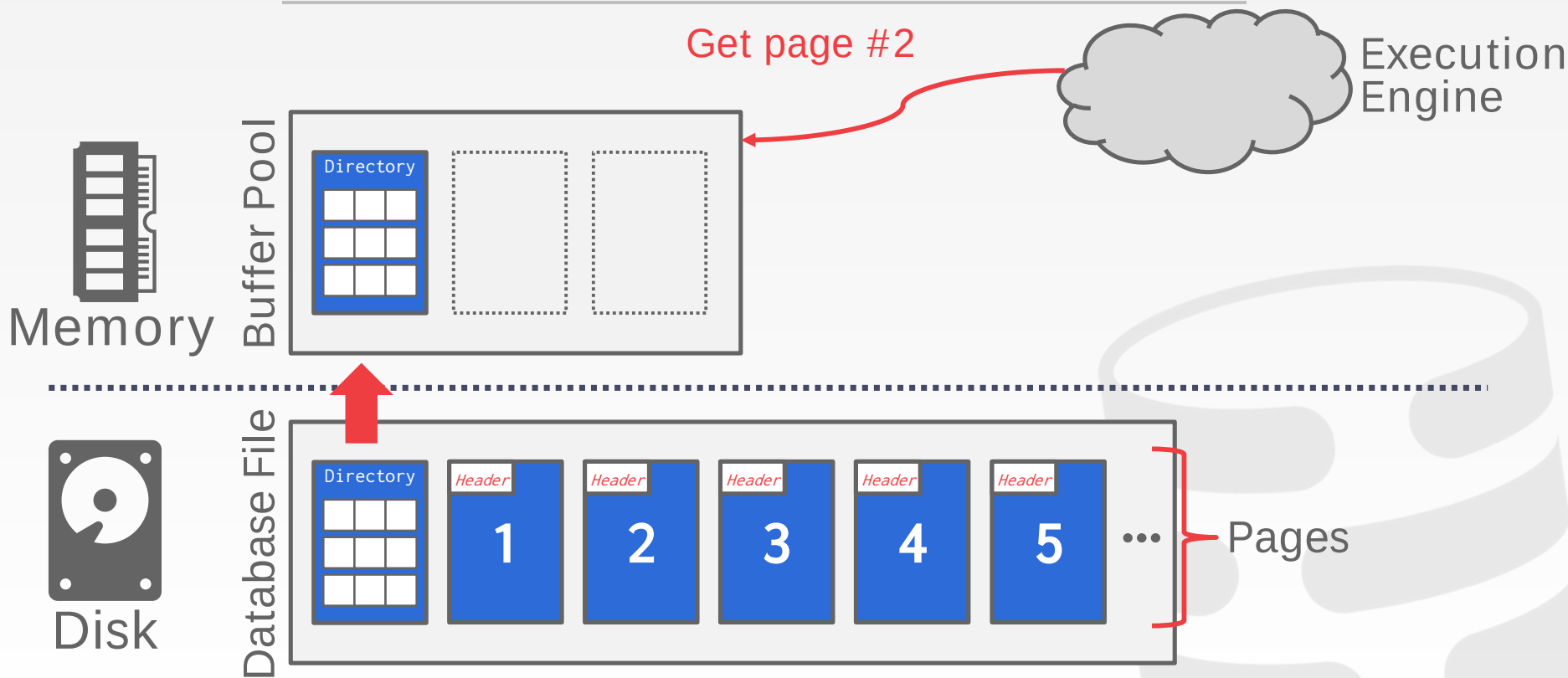
DISK-ORIENTED DBMS



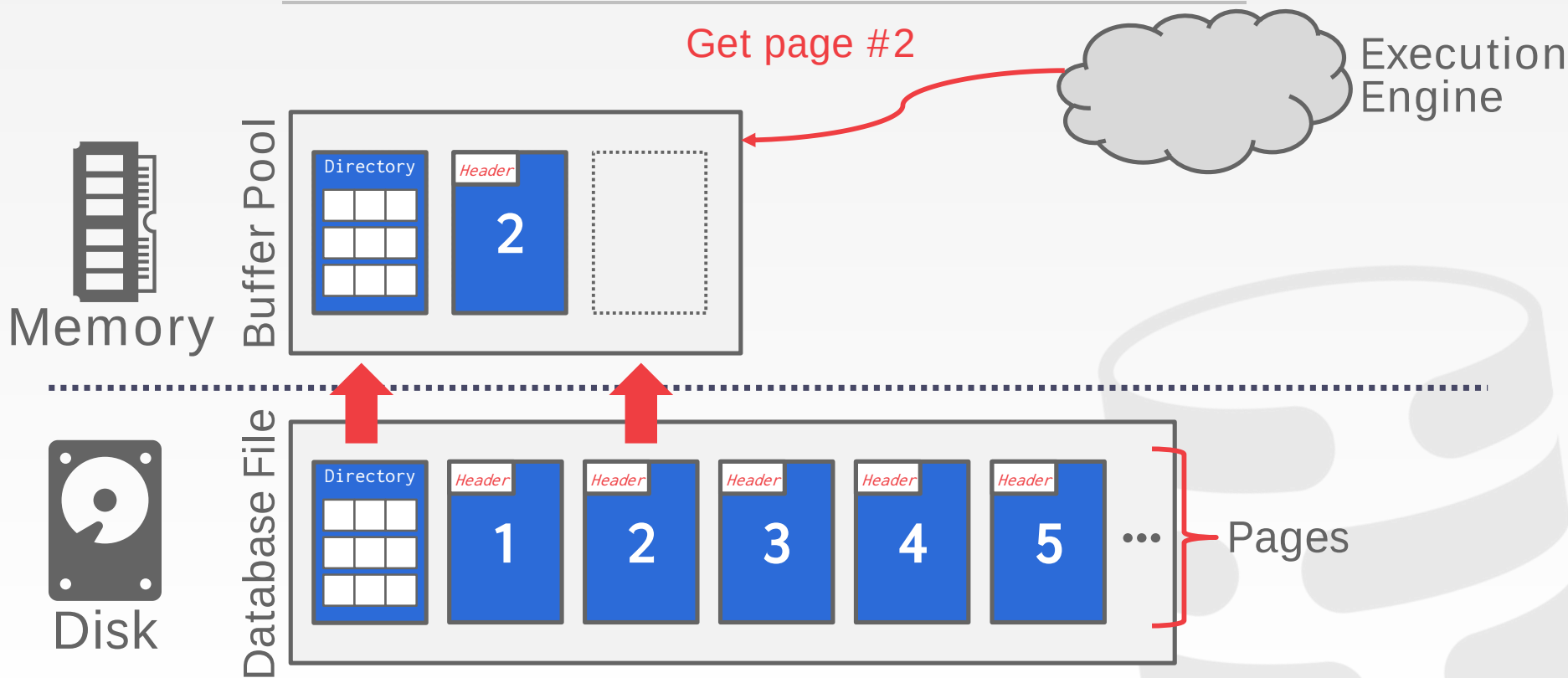
DISK-ORIENTED DBMS



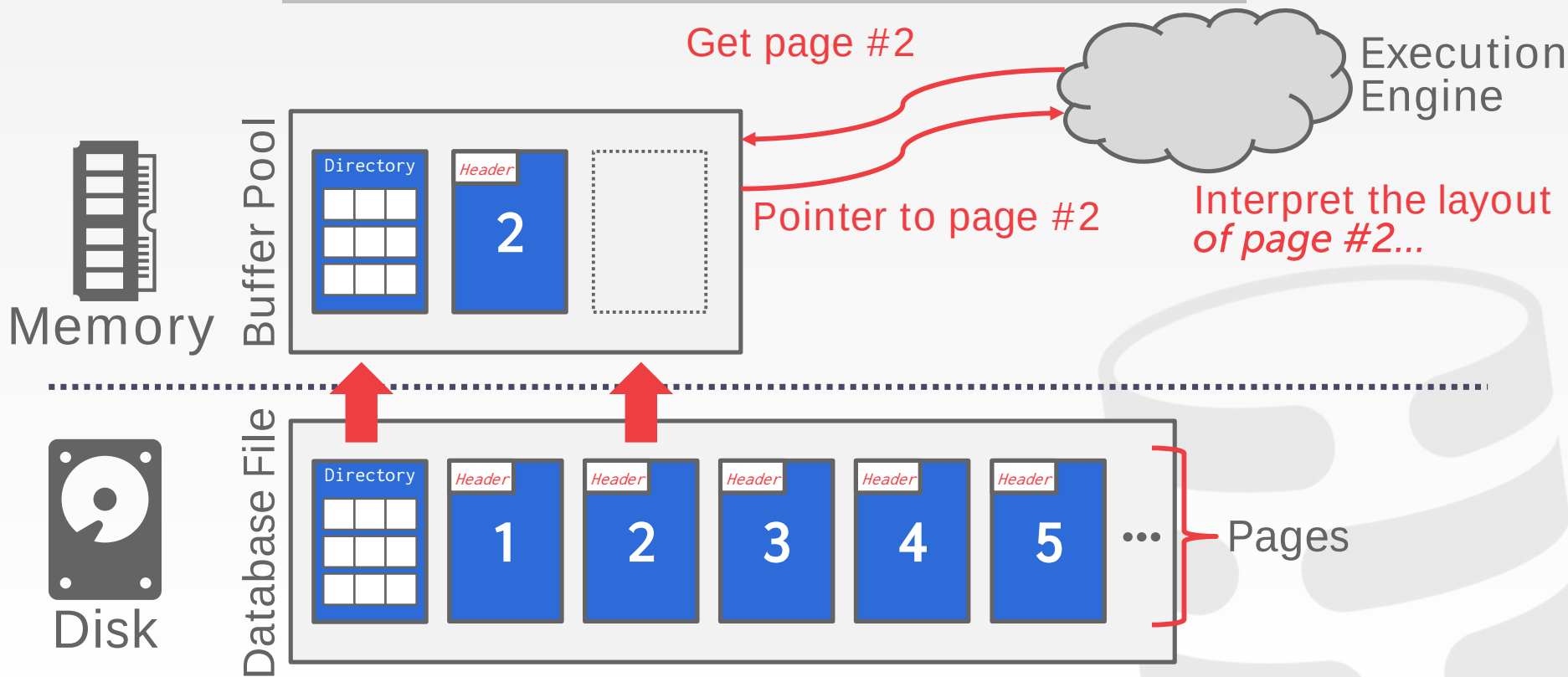
DISK-ORIENTED DBMS



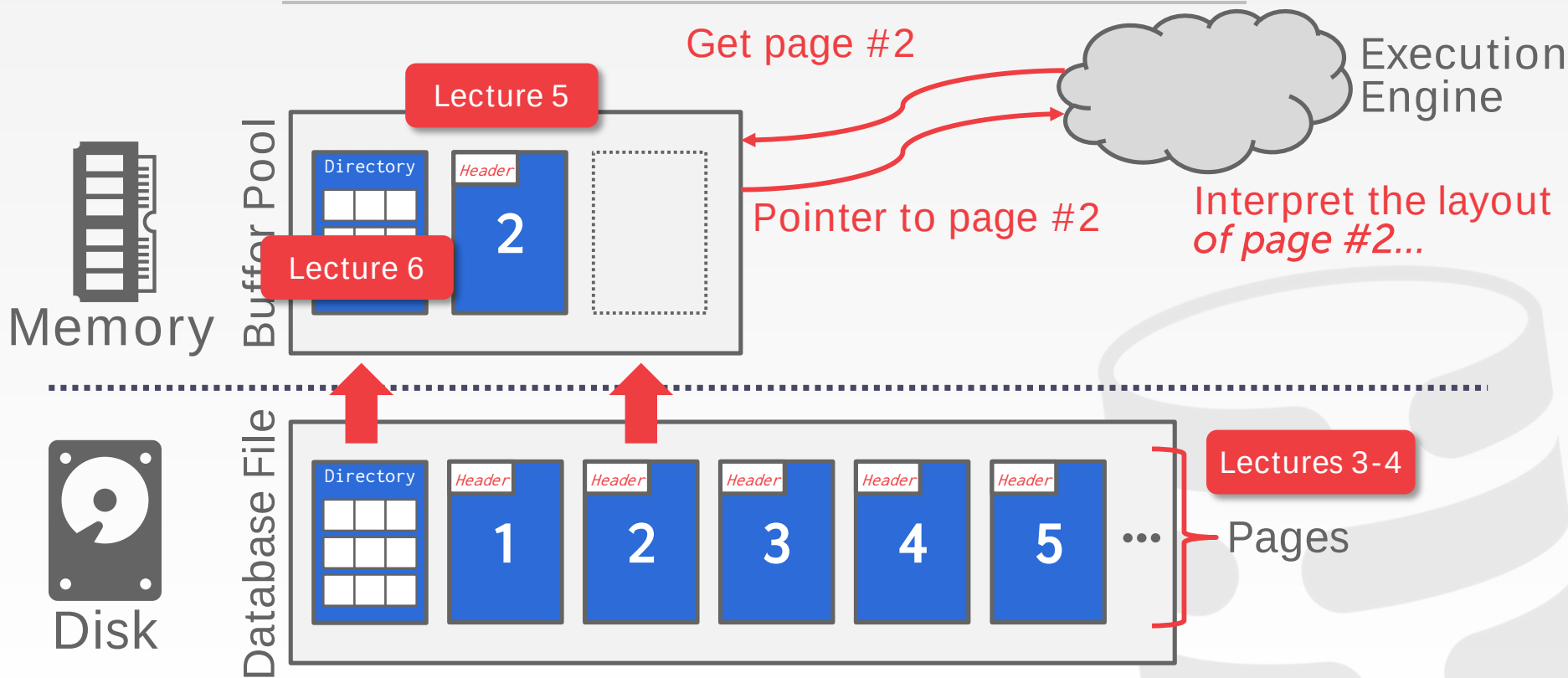
DISK-ORIENTED DBMS



DISK-ORIENTED DBMS



DISK-ORIENTED DBMS

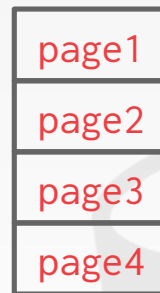


WHY NOT USE THE OS?

One can use memory mapping (**mmap**) to store the contents of a file into a process' address space.

The OS is responsible for moving data for moving the files' pages in and out of memory.

Virtual
Memory



Physical
Memory

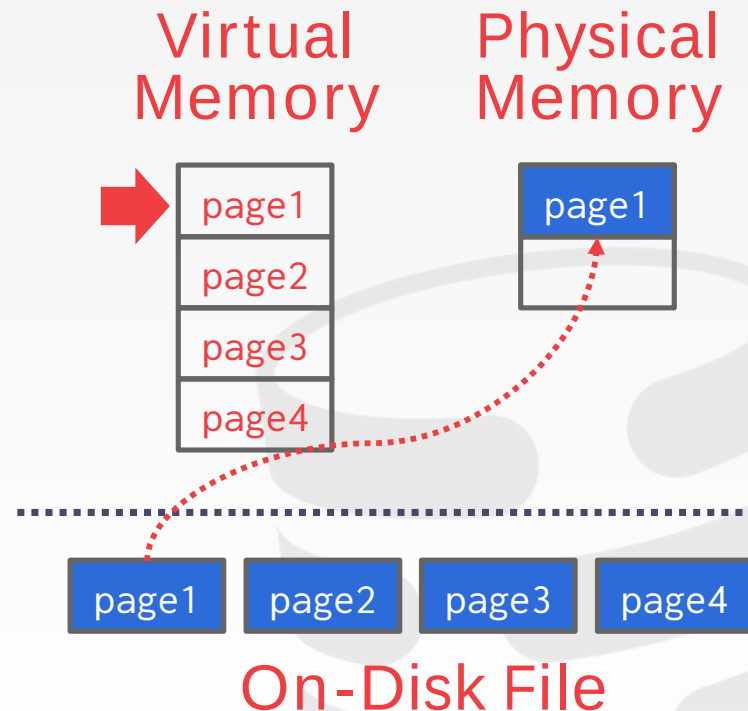


On-Disk File

WHY NOT USE THE OS?

One can use memory mapping (**mmap**) to store the contents of a file into a process' address space.

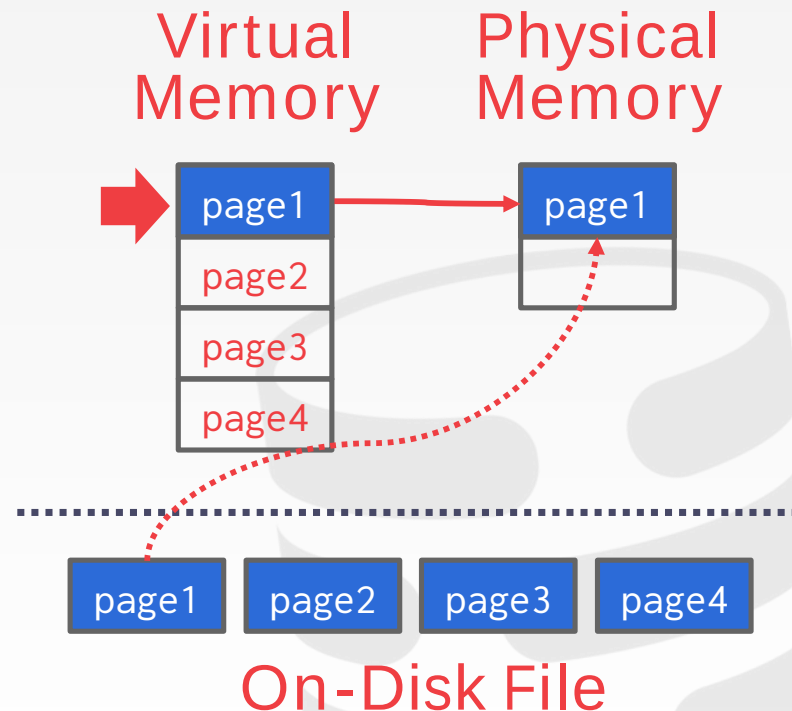
The OS is responsible for moving data for moving the files' pages in and out of memory.



WHY NOT USE THE OS?

One can use memory mapping (**mmap**) to store the contents of a file into a process' address space.

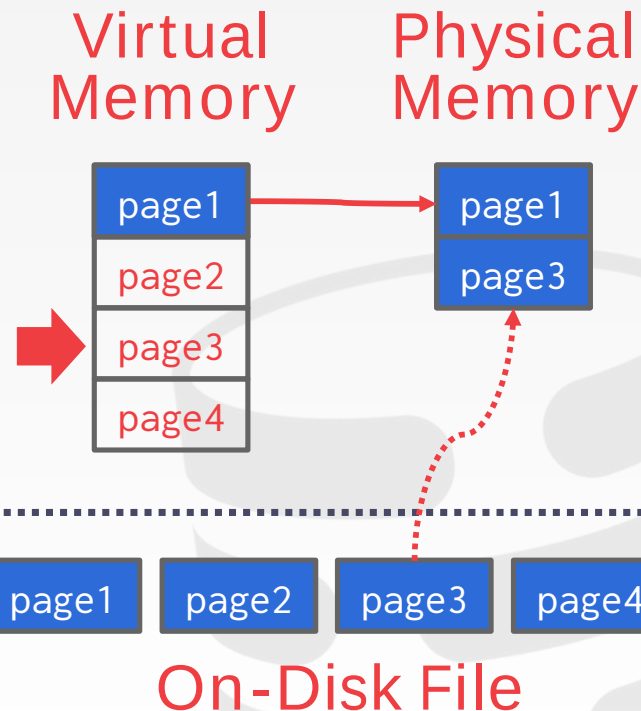
The OS is responsible for moving data for moving the files' pages in and out of memory.



WHY NOT USE THE OS?

One can use memory mapping (**mmap**) to store the contents of a file into a process' address space.

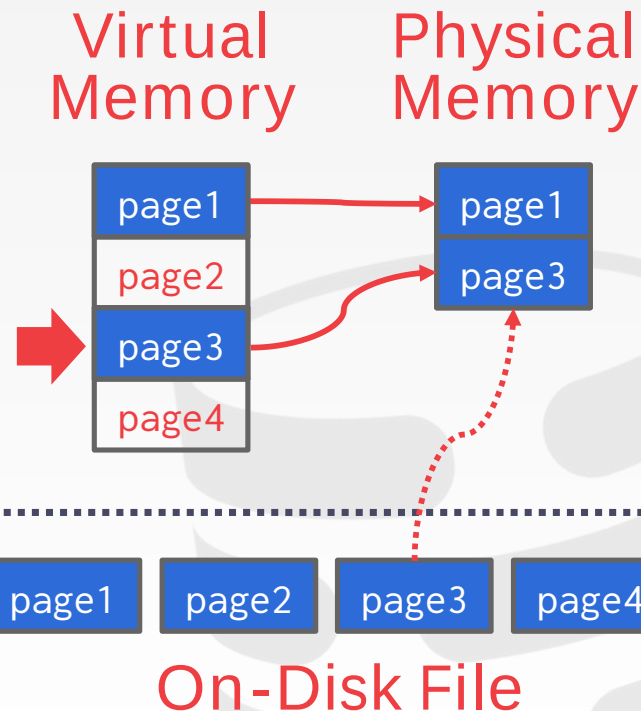
The OS is responsible for moving data for moving the files' pages in and out of memory.



WHY NOT USE THE OS?

One can use memory mapping (**mmap**) to store the contents of a file into a process' address space.

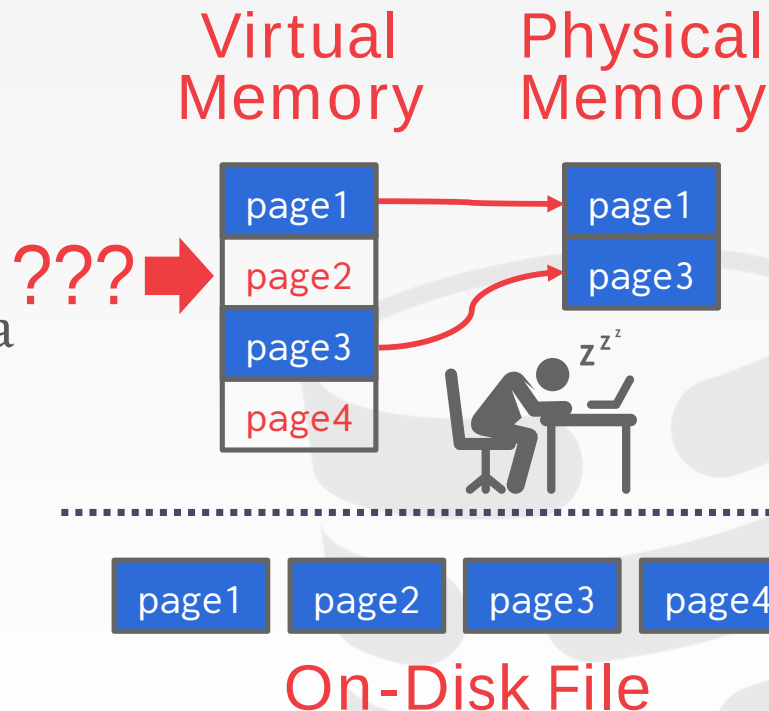
The OS is responsible for moving data for moving the files' pages in and out of memory.



WHY NOT USE THE OS?

One can use memory mapping (**mmap**) to store the contents of a file into a process' address space.

The OS is responsible for moving data for moving the files' pages in and out of memory.



WHY NOT USE THE OS?

What if we allow multiple threads to access the **mmap** files to hide page fault stalls?

This works good enough for read-only access.
It is complicated when there are multiple writers...

WHY NOT USE THE OS?

There are some solutions to this problem:

- **advise**: Tell the OS how you expect to read certain pages.
- **mlock**: Tell the OS that memory ranges cannot be paged out.
- **msync**: Tell the OS to flush memory ranges out to disk.

Full Usage



Partial Usage



mongoDB



SQLite



influxdb

WHY NOT USE THE OS?

DBMS (almost) always wants to control things itself and can do a better job at it.

- Flushing dirty pages to disk in the correct order.
- Specialized prefetching.
- Buffer replacement policy.
- Thread/process scheduling.

The OS is **not** your friend.



DATABASE STORAGE

Problem #1: How the DBMS represents the database in files on disk.

← Today

Problem #2: How the DBMS manages its memory and move data back-and-forth from disk.

TODAY'S AGENDA

File Storage

Page Layout

Tuple Layout



FILE STORAGE

The DBMS stores a database as one or more files on disk.

→ The OS doesn't know anything about the contents of these files.

Early systems in the 1980s used custom filesystems on raw storage.

→ Some "enterprise" DBMSs still support this.

→ Most newer DBMSs do not do this.

STORAGE MANAGER

The storage manager is responsible for maintaining a database's files.

→ Some do their own scheduling for reads and writes to improve spatial and temporal locality of pages.

It organizes the files as a collection of pages.

→ Tracks data read/written to pages.

→ Tracks the available space.

DATABASE PAGES

A page is a fixed-size block of data.

- It can contain tuples, meta-data, indexes, log records...
- Most systems do not mix page types.
- Some systems require a page to be self-contained.

Each page is given a unique identifier.

- The DBMS uses an indirection layer to map page ids to physical locations.

DATABASE PAGES

There are three different notions of "pages" in a DBMS:

- Hardware Page (usually 4KB)
- OS Page (usually 4KB)
- Database Page (512B-16KB)

By hardware page, we mean at what level the device can guarantee a "failsafe write".

4KB



ORACLE®

8KB



16KB



PAGE STORAGE ARCHITECTURE

Different DBMSs manage pages in files on disk in different ways.

- Heap File Organization
- Sequential / Sorted File Organization
- Hashing File Organization

At this point in the hierarchy we don't need to know anything about what is inside of the pages.

DATABASE HEAP

A heap file is an unordered collection of pages where tuples that are stored in random order.

- Create / Get / Write / Delete Page
- Must also support iterating over all pages.

Need meta-data to keep track of what pages exist and which ones have free space.

Two ways to represent a heap file:

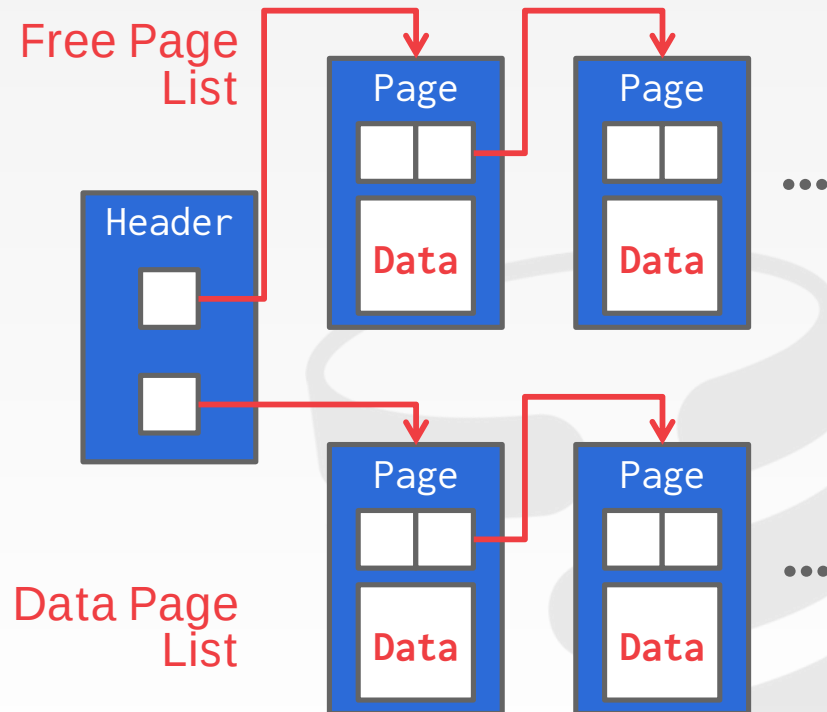
- Linked List
- Page Directory

HEAP FILE: LINKED LIST

Maintain a header page at the beginning of the file that stores two pointers:

- HEAD of the free page list.
- HEAD of the data page list.

Each page keeps track of the number of free slots in itself.

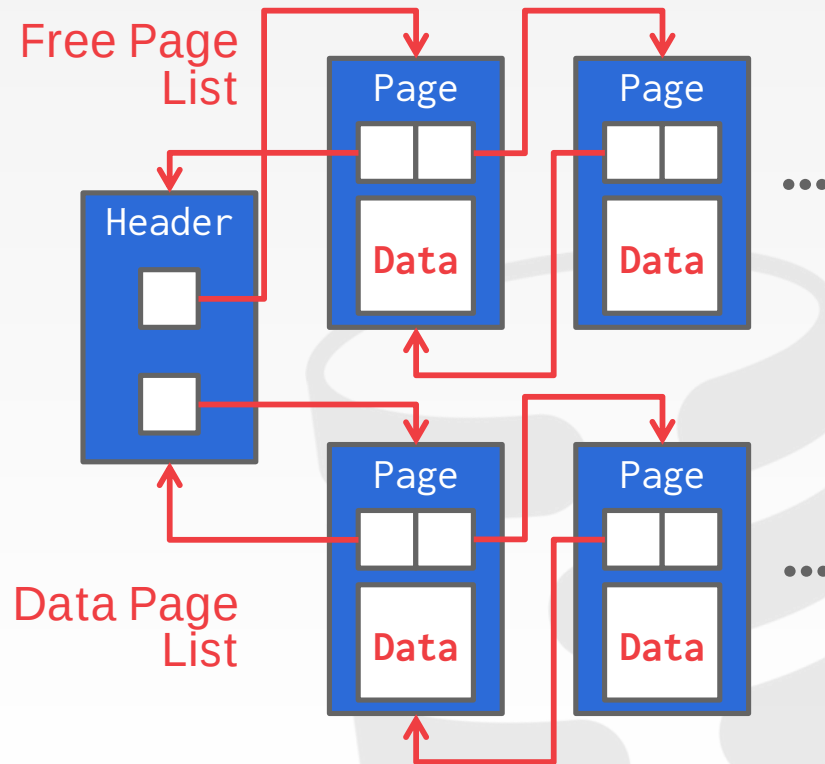


HEAP FILE: LINKED LIST

Maintain a header page at the beginning of the file that stores two pointers:

- HEAD of the free page list.
- HEAD of the data page list.

Each page keeps track of the number of free slots in itself.

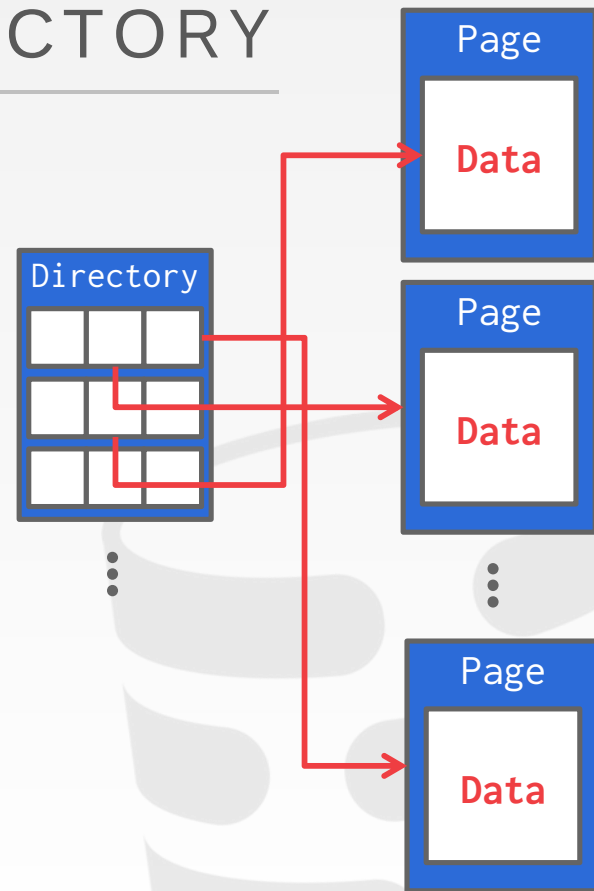


HEAP FILE: PAGE DIRECTORY

The DBMS maintains special pages that track the location of data pages in the database files.

The directory also records the number of free slots per page.

The DBMS has to make sure that the directory pages are in sync with the data pages.



TODAY'S AGENDA

File Storage

Page Layout

Tuple Layout

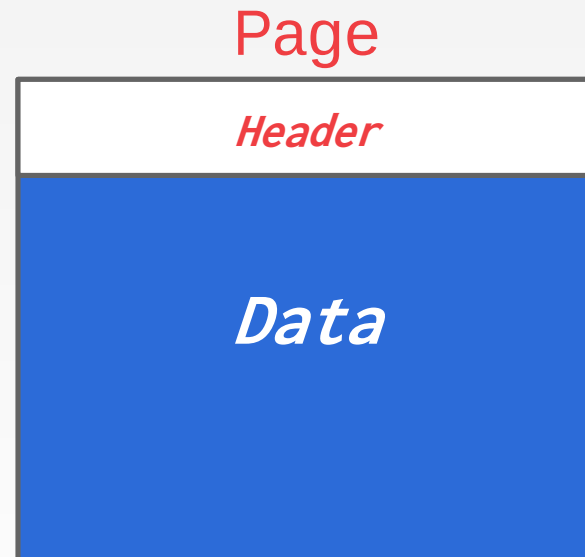


PAGE HEADER

Every page contains a header of meta-data about the page's contents.

- Page Size
- Checksum
- DBMS Version
- Transaction Visibility
- Compression Information

Some systems require pages to be self-contained (e.g., Oracle).



PAGE LAYOUT

For any page storage architecture, we now need to understand how to organize the data stored inside of the page.

→ We are still assuming that we are only storing tuples.

Two approaches:

→ Tuple-oriented

→ Log-structured

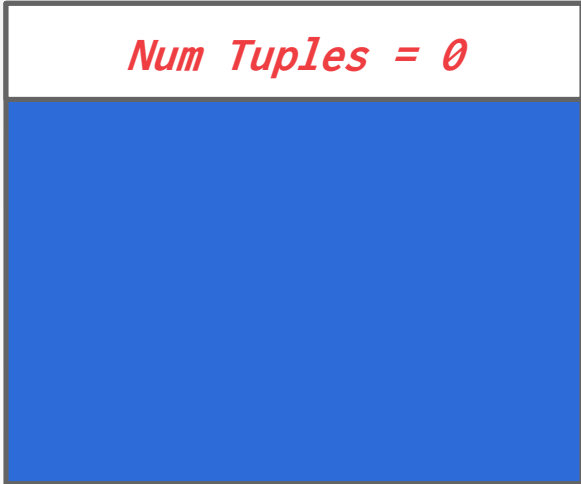


TUPLE STORAGE

How to store tuples in a page?

Page

Num Tuples = 0



TUPLE STORAGE

How to store tuples in a page?

Strawman Idea: Keep track of the number of tuples in a page and then just append a new tuple to the end.

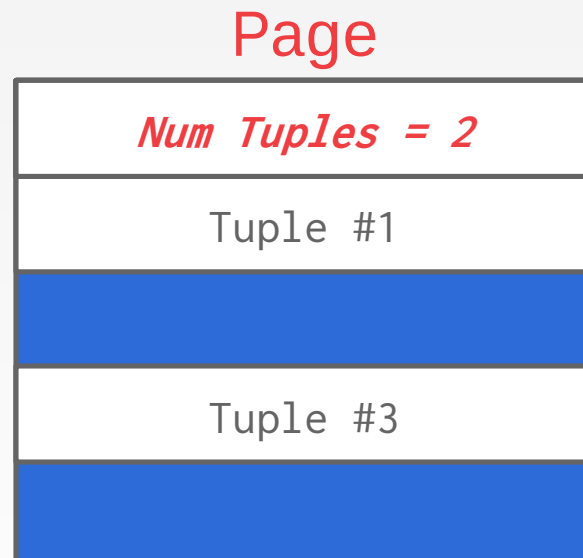
Page

<i>Num Tuples = 3</i>
Tuple #1
Tuple #2
Tuple #3

TUPLE STORAGE

How to store tuples in a page?

Strawman Idea: Keep track of the number of tuples in a page and then just append a new tuple to the end.
→ What happens if we delete a tuple?



TUPLE STORAGE

How to store tuples in a page?

Strawman Idea: Keep track of the number of tuples in a page and then just append a new tuple to the end.
→ What happens if we delete a tuple?

Page

<i>Num Tuples = 3</i>
Tuple #1
Tuple #4
Tuple #3

TUPLE STORAGE

How to store tuples in a page?

Strawman Idea: Keep track of the number of tuples in a page and then just append a new tuple to the end.

- What happens if we delete a tuple?
- What happens if we have a variable-length attribute?

Page

<i>Num Tuples = 3</i>
Tuple #1
Tuple #4
Tuple #3

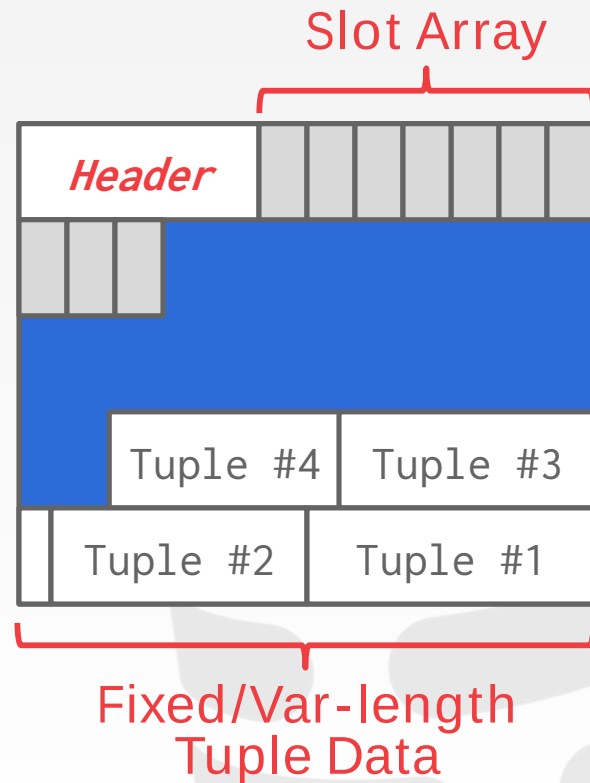
SLOTTED PAGES

The most common layout scheme is called slotted pages.

The slot array maps "slots" to the tuples' starting position offsets.

The header keeps track of:

- The # of used slots
- The offset of the starting location of the last slot used.



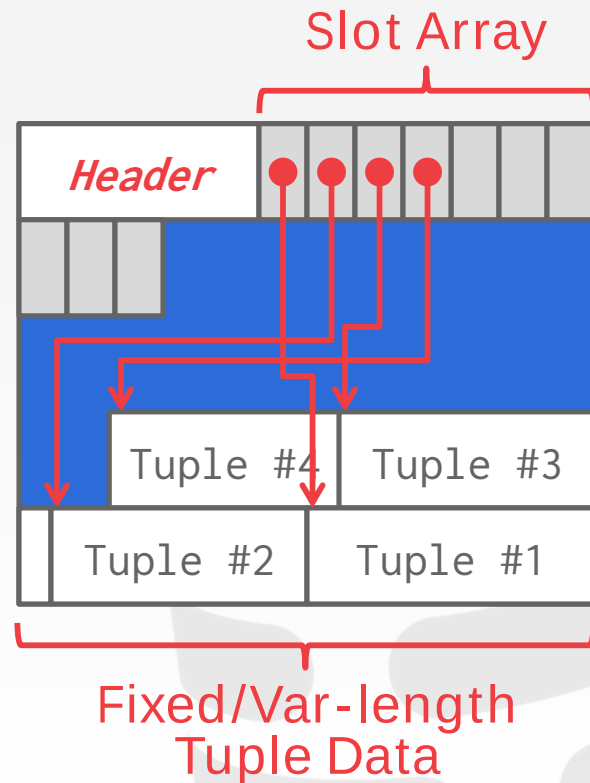
SLOTTED PAGES

The most common layout scheme is called slotted pages.

The slot array maps "slots" to the tuples' starting position offsets.

The header keeps track of:

- The # of used slots
- The offset of the starting location of the last slot used.



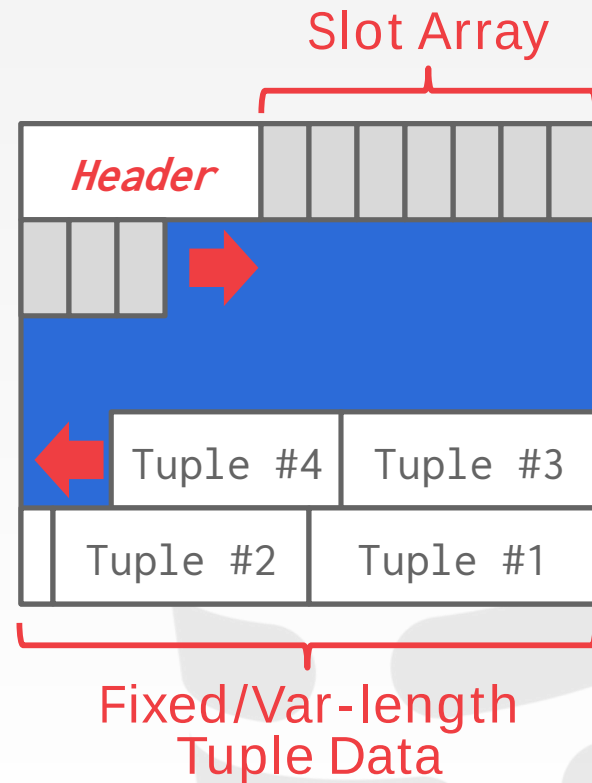
SLOTTED PAGES

The most common layout scheme is called slotted pages.

The slot array maps "slots" to the tuples' starting position offsets.

The header keeps track of:

- The # of used slots
- The offset of the starting location of the last slot used.

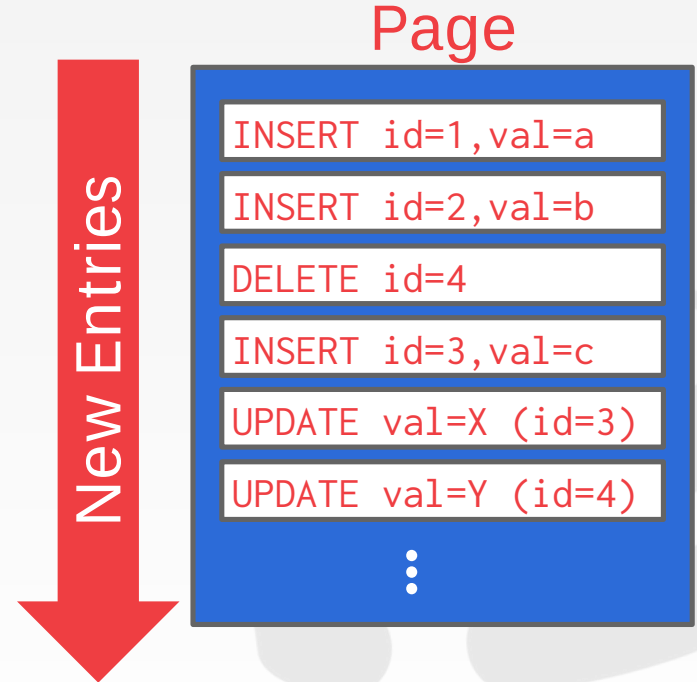


LOG-STRUCTURED FILE ORGANIZATION

Instead of storing tuples in pages, the DBMS only stores log records.

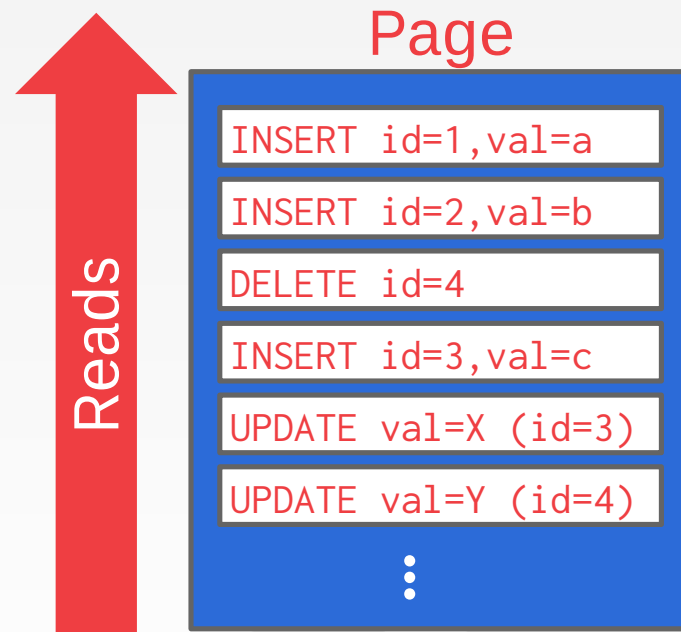
The system appends log records to the file of how the database was modified:

- Inserts store the entire tuple.
- Deletes mark the tuple as deleted.
- Updates contain the delta of just the attributes that were modified.



LOG-STRUCTURED FILE ORGANIZATION

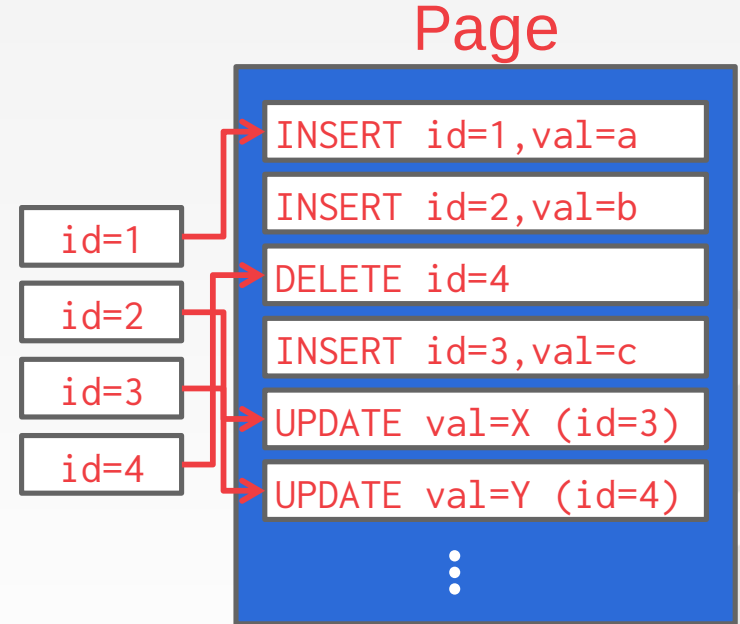
To read a record, the DBMS scans the log backwards and "recreates" the tuple to find what it needs.



LOG-STRUCTURED FILE ORGANIZATION

To read a record, the DBMS scans the log backwards and "recreates" the tuple to find what it needs.

Build indexes to allow it to jump to locations in the log.



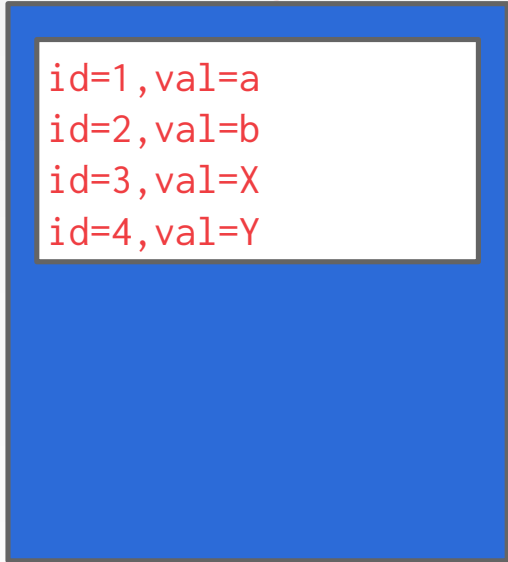
LOG-STRUCTURED FILE ORGANIZATION

To read a record, the DBMS scans the log backwards and "recreates" the tuple to find what it needs.

Build indexes to allow it to jump to locations in the log.

Periodically compact the log.

Page



```
id=1, val=a  
id=2, val=b  
id=3, val=X  
id=4, val=Y
```

APACHE
HBASE



levelDB



RocksDB

TODAY'S AGENDA

File Storage

Page Layout

Tuple Layout



TUPLE LAYOUT

A tuple is essentially a sequence of bytes.

It's the job of the DBMS to interpret those bytes into attribute types and values.



TUPLE HEADER

Each tuple is prefixed with a header that contains meta-data about it.

- Visibility info (concurrency control)
- Bit Map for **NULL** values.

We do not need to store meta-data about the schema.

Tuple



TUPLE DATA

Attributes are typically stored in the order that you specify them when you create the table.

This is done for software engineering reasons.

We re-order attributes automatically in CMU's new DBMS...

Tuple

<i>Header</i>	a	b	c	d	e
---------------	---	---	---	---	---

```
CREATE TABLE foo (  
  a INT PRIMARY KEY,  
  b INT NOT NULL,  
  c INT,  
  d DOUBLE,  
  e FLOAT  
);
```

DENORMALIZED TUPLE DATA

Can physically *denormalize* (e.g., "pre join") related tuples and store them together in the same page.

- Potentially reduces the amount of I/O for common workload patterns.
- Can make updates more expensive.

```
CREATE TABLE foo (  
  a INT PRIMARY KEY,  
  b INT NOT NULL,  
);
```

```
CREATE TABLE bar (  
  c INT PRIMARY KEY,  
  a INT  
  REFERENCES foo (a),  
);
```



DENORMALIZED TUPLE DATA

Can physically *denormalize* (e.g., "pre join") related tuples and store them together in the same page.

- Potentially reduces the amount of I/O for common workload patterns.
- Can make updates more expensive.

foo

<i>Header</i>	a	b
---------------	---	---

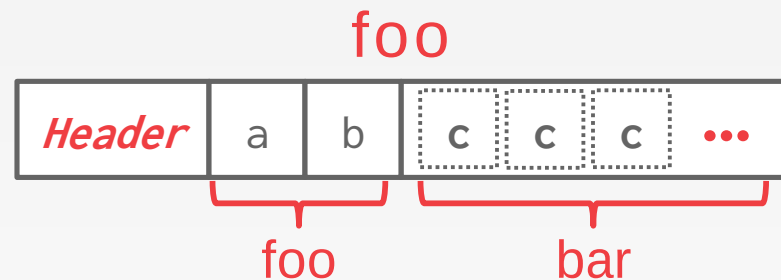
bar

<i>Header</i>	c	a
<i>Header</i>	c	a
<i>Header</i>	c	a

DENORMALIZED TUPLE DATA

Can physically *denormalize* (e.g., "pre join") related tuples and store them together in the same page.

- Potentially reduces the amount of I/O for common workload patterns.
- Can make updates more expensive.



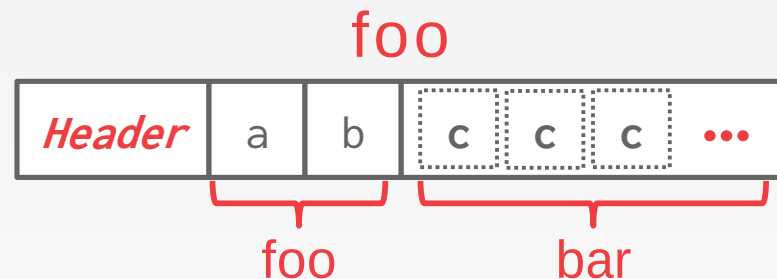
DENORMALIZED TUPLE DATA

Can physically *denormalize* (e.g., "pre join") related tuples and store them together in the same page.

- Potentially reduces the amount of I/O for common workload patterns.
- Can make updates more expensive.

Not a new idea.

- IBM System R did this in the 1970s.
- Several NoSQL DBMSs do this without calling it physical denormalization.



RECORD IDS

The DBMS needs a way to keep track of individual tuples.

Each tuple is assigned a unique record identifier.

- Most common: **page_id** + **offset/slot**
- Can also contain file location info.

An application cannot rely on these ids to mean anything.



CTID (4-bytes)



ROWID (8-bytes)

ORACLE[®]

ROWID (10-bytes)

CONCLUSION

Database is organized in pages.

Different ways to track pages.

Different ways to store pages.

Different ways to store tuples.



NEXT CLASS

Value Representation
Storage Models



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]